Szoftverfejlesztő és -tesztelő

Project Lynqo

Készítette
André Norbert
Varga Bianka
Kocsis Róbert Benjámin
Szoftverfejlesztő és -tesztelő szak

Témavezető
Kerényi Róbert Nándor
oktató

Miskolc, 2026

Tartalomjegyzék

Bevezetés 4

1. Technológiák **5**

1.1. Használt Technológiák 5

1.1.1. Backend 6

1.1.2. Frontend 7

1.1.3. Fejlesztői Eszközök 8

1.2. Projekt Felépítése 8

1.3. JwtToken Generátor Funkció 9

1.3.1. JwtHelper Osztály 9

1.3.2. UpdateMyProfile Végpont 10

1.3.3. Felhasználói Élmény 11

2. Relációs adatkezelő rendszerek **12**

2.1. Relációs adatbázis 12

2.1.1. A relációs adatbáziskezelő rendszerek 12

2.1.2. MySQL és MariaDB, valamint PHPMyAdmin 14

3. React Fejlesztői Környezet **16**

3.1. Bevezetés 16

3.2. React Fejlesztői Környezet 16

3.2.1. React Telepítése és Projektindítás 17

3.2.2. Fejlesztői Eszközök 19

3.3. React Alapjai 19

3.3.1. Komponensek és JSX 20

3.3.2. Állapotkezelés a React-ben 21

3.3.3. Props 23

3.3.4. Komponensek Típusai 23

3.4. Profil Adatok Megváltoztatása 24

3.4.1. Payload Komponens 25

3.4.2. Új Profil Adatok 26

3.4.3. Felhasználói Élmény 26

3.5. Összefoglalás 26

Irodalomjegyzék **28**

Bevezetés

A Lynqo egy modern, innovatív nyelvtanuló alkalmazás, kicsik és nagyok számára egyaránt hatékony segítséget nyújt az idegen nyelvek elsajátításában. Játékos, mégis strukturált módon vezeti be a felhasználókat a kiválasztott nyelv alapjaiba, valamint lehetőséget biztosítson a már meglévő nyelvtudás továbbfejlesztésére is.

Különös hangsúlyt fektetünk a könnyen kezelhető, letisztult felhasználói felületre és a gyors, motiváló tanulási folyamatra. A rendszer interaktív feladatokon, szókincsfejlesztő és nyelvtani gyakorlatokon keresztül segíti a nyelvtanulást. Az alkalmazás több választható nyelvet is kínál, mint például angol és francia nyelvi modulokkal, lehetővé tesszük a felhasználók számára, hogy saját tempójukban fejlődjenek és akár hosszú távon fenntartható motivációt biztosítsunk minden korosztály számára.

1. fejezet

Technológiák

1.1. Használt Technológiák

A Lynqo platform megalkotásakor a legmodernebb szoftverfejlesztési eljárásokat alkalmaztuk az architektúra összes szintjén, így a háttérrendszer, az adatbázis és a felhasználói felület is naprakész alapokon nyugszik.

A szerveroldali infrastruktúra motorjaként a Node.js futtatókörnyezetet választottuk. Ennek a technológiának hála kiemelkedő sebességű, aszinkron adatkezelést és rugalmasan bővíthető API végpontokat tudtunk létrehozni. Ez a backend réteg menedzseli a biztonságos bejelentkezést (autentikációt), futtatja a platform üzleti logikáját, és garantálja a védett adatcserét az adatbázissal.

A felhasználói felület (frontend) a React könyvtárra épül, ami folyamatos, interaktív és minden kijelzőn tökéletesen mutató (reszponzív) élményt nyújt. Az építőkockákból álló, komponensalapú struktúra jelentősen felgyorsítja a programozási munkát, és hosszú távon is átláthatóvá teszi a forráskódot. A letisztult dizájn és a prémium felhasználói élmény (UX) elérése érdekében olyan korszerű UI csomagokat (mint a Material-UI vagy a Bootstrap) és CSS preprocesszorokat vetünk be, amelyek vizuálisan is magasabb szintre emelik az oldalt.

Ami az adattárolást illeti, egy MySQL relációs adatbázis-kezelő felel a rekordok logikus elrendezéséért és villámgyors lekérdezéséért. A szerver és az adatbázis közti kommunikációt közvetlen SQL utasításokkal oldottuk meg, ami tiszta és rendkívül hatékony adatmozgást eredményez a logikai rétegben. A kész alkalmazás élesítése (deployment) pedig a Microsoft Azure felhőkörnyezetében történik, ami vállalati szintű stabilitást, dinamikus skálázhatóságot és maximális biztonságot garantál a Lynqo számára.

1.1.1. Backend

A platform szerveroldali logikáját a nyílt forráskódú és teljesen platformfüggetlen .NET Core keretrendszerre építettük fel. Ez a modern technológiai háttér tökéletes alapot ad a felhőre optimalizált, villámgyors és könnyedén bővíthető alkalmazások létrehozásához. Kiemelkedő rugalmasságát mutatja, hogy megkötések nélkül futtatható Windows, Linux vagy akár macOS operációs rendszereken is.

A fejlesztés során maximálisan kihasználjuk, hogy a keretrendszer kiválóan támogatja a mikroszolgáltatás-alapú (microservices) architektúrát. Ennek köszönhetően a komplex rendszert kisebb, önállóan is működő és jól szeparált modulokra tudtuk bontani. Ez a megközelítés nemcsak a forráskód karbantartását teszi átláthatóbbá, de garantálja a folyamatos rendelkezésre állást és a dinamikus skálázhatóságot is.

Az adatbázis-műveleteket és a rekordok hatékony menedzselését az Entity Framework Core végzi, jelentősen leegyszerűsítve az adatelérést. Emellett a technológia gyors és stabil RESTful web API-k kialakítását is lehetővé teszi, ami hibátlan és zökkenőmentes adatcserét biztosít a felhasználói felület (frontend) és a szerver (backend) között.

ASP.NET Core

A platform webes alkalmazásainak és API-jainak motorját a .NET ökoszisztémába tartozó ASP.NET Core adja. Ez a sokoldalú keretrendszer kiemelkedő sebességet, operációs rendszereken átívelő (cross-platform) működést és rugalmas, moduláris architektúrát garantál. Teljes mértékben igazodik a modern szoftverfejlesztési elvárásokhoz: natívan támogatja a függőséginjektálást (dependency injection), a köztesrétegek (middlewareek) használatát és a letisztult RESTful API-k felépítését.

A technológia másik nagy erőssége, hogy az így megírt szolgáltatások telepítése és erőforrás-bővítése rendkívül gördülékeny, ami ideális választássá teszi a felhőalapú (cloud) környezetekhez. Ezzel a fejlett eszköztárral a programozók hatékonyan, mégis gyorsan hozhatnak létre olyan stabil rendszereket, amelyek a legszigorúbb biztonsági és megbízhatósági követelményeknek is maradéktalanul eleget tesznek.

platform adatainak biztonságos és strukturált tárolását egy MySQL adatbázis végzi. Ez az iparági standardnak számító, nyílt forráskódú relációs technológia kiváló teljesítményt és maximális stabilitást garantál a háttérben. Az adminisztrációs feladatok és a rekordok menedzselése a phpMyAdmin szoftveren keresztül történik, amely egy kényelmes, böngészőből vezérelhető grafikus környezetet ad a rendszer kezeléséhez.

1.1.2. Frontend

A dinamikus és interaktív felhasználói felületek (UI) megalkotásának egyik legkedveltebb eszköze a React, egy széles körben használt JavaScript-könyvtár. Lényege a komponensalapú felépítés: a programozók olyan önálló építőkockákat hozhatnak létre, amelyek mindegyike saját belső adatokkal és funkciókkal rendelkezik. A rendszer villámgyors megjelenítését a virtuális DOM garantálja, amely a valós DOM felesleges és lassú módosításainak kiiktatásával pörgeti fel a teljesítményt. Az így megírt szoftverek rugalmasan skálázhatók és könnyen menedzselhetők, a fejlesztést pedig olyan népszerű kiegészítők segítik, mint az állapotkezelő Redux vagy a navigációért felelős React Router. A technológia ráadásul a weben is túlmutat: a React Native bevonásával akár mobilapplikációkat is készíthetünk ugyanerre a logikára építve.

React

React egy olyan JavaScript-könyvtár, amely az építőkockákra – azaz komponensekre – épülő architektúrát követi. Legfőbb küldetése, hogy zökkenőmentesebbé és gyorsabbá tegye a felhasználói felületek megalkotását. A programozók olyan önálló, logikai egységeket készíthetnek a segítségével, amelyeket később újra és újra fel tudnak használni, így az elkészült szoftver tisztább, könnyebben karbantartható és jól továbbfejleszthető marad.

Kiemelkedő technológiai újítása a virtuális DOM alkalmazása. Ez a megoldás azzal pörgeti fel az alkalmazások sebességét, hogy csak a legszükségesebb esetekben nyúl hozzá a tényleges DOM-hoz, elkerülve a lassító és felesleges módosításokat. Emellett a keretrendszer a JSX formátumra építkezik, amivel a HTML-hez nagyon hasonló struktúrákat ágyazhatunk közvetlenül a JavaScriptbe. Ez a megközelítés sokkal logikusabbá és átláthatóbbá teszi a forráskódot.

A React köré egy hatalmas és gazdag ökoszisztéma épült, amely rengeteg azonnal bevethető kiegészítő csomagot kínál. Ilyen például a kliensoldali navigációért és az oldalak közötti váltásért felelős React Router, vagy az összetett adatok kezelését (state) egyben tartó Redux. Ezekkel a professzionális eszközökkel a fejlesztők még robusztusabb, kiválóan skálázható és stabil platformokat tudnak felépíteni.

1.1.3. Fejlesztői Eszközök

A fejlesztés során különböző eszközöket használunk a hatékonyság növelése és a hibakeresés megkönnyítése érdekében:

- **PHPMyAdmin**: Egy webalapú eszköz a MySQL adatbázisok kezelésére.
- **React Developer Tools**: Egy böngészőbővítmény, amely lehetővé teszi a React komponensek és állapotok ellenőrzését a böngészőben.
- **Visual Studio Code (VSCode)**: Egy népszerű kódszerkesztő, amely számos bővítménnyel rendelkezik, például az `ESLint` és `Prettier` segítségével.

1.2. Projekt Felépítése

A Lynqo projekt három fő komponensből áll: backend, frontend és adatbázis. Az alábbiakban bemutatjuk ezeknek a komponenseknek a felépítését és funkcióit.

1.3. JwtToken Generátor Funkció

A Project Lynqo backend tartalmaz egy Token generátort ami garantálja a felhasználó bejelentkezve maradását az oldalon, biztosítva a biztonságos és zökkenőmentes felhasználói élményt.

1.3.1. JwtHelper Osztály

A `JwtHelper` osztály felelős a Jwt Token generálásáért.

Az alábbi kód egy részletet mutat be a `GenerateJwtToken` metódusból:

```
public static string GenerateJwtToken(User user, string secretKey, string issuer, string audience, int expiresMinutes = 60)
        {
            var claims = new[]
            {
            new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
            new Claim(JwtRegisteredClaimNames.UniqueName, user.Username),
            new Claim("role", user.Role)
        };

            var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secretKey));
            var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

            var token = new JwtSecurityToken(
                issuer: issuer,
                audience: audience,
                claims: claims,
                expires: DateTime.UtcNow.AddMinutes(expiresMinutes),
                signingCredentials: creds);

            return new JwtSecurityTokenHandler().WriteToken(token);
        }
```

1.3.2. UpdateMyProfile Végpont

A `ProfileController` egy API végpontot biztosít. A végpont a következőképpen működik:

- **POST /api/me**: Ez a végpont fogadja a felhasználó előzőleg megadott adatait és lehetővé teszi azoknak a változtatását.

  Az alábbi kód egy részletet mutat be a végpont implementációjából:

```
[HttpPut("me")]
[Authorize]
public async Task<IActionResult> UpdateMyProfile([FromBody]
    UpdateProfileDto dto)
{
    try
    {
        var userId = GetCurrentUserId();
        if (userId == null)
            return Unauthorized(new { message = "Invalid
                token." });

        var user = await _context.Users.FindAsync(userId.
            Value);
        if (user == null)
            return NotFound(new { message = "User not found
                ." });

        if (!string.IsNullOrWhiteSpace(dto.Username))
            user.Username = dto.Username.Trim();

        if (!string.IsNullOrWhiteSpace(dto.DisplayName))
            user.DisplayName = dto.DisplayName.Trim();

        if (!string.IsNullOrWhiteSpace(dto.Email))
            user.Email = dto.Email.Trim();

        if (!string.IsNullOrWhiteSpace(dto.Password))
        {
            user.PasswordHash = PasswordHasher.HashPassword
                (dto.Password);
        }

        await _context.SaveChangesAsync();
```

```
30
31          var profileData = await BuildProfileDataAsync(user)
                ;
32          return Ok(profileData);
33      }
34      catch (Exception ex)
35      {
36          return StatusCode(500, new
37          {
38              error = "Crash in PUT /me",
39              details = ex.InnerException?.Message ?? ex.
                    Message
40          });
41      }
42  }
```

1.3.3. Felhasználói Élmény

A te személyes tanulási útvonalad A platformunk biztosítja, hogy egy letisztult és könnyen áttekinthető tanulási útvonalon (ösvényen) haladhass. Nem kell időpontokhoz igazodnod: egyből rálátsz a napi céljaidra, a következő kihívásokra és az aktuális szintedre, legyen szó angolról, spanyolról vagy franciáról. A rendszer logikusan építi fel a leckéket, figyelembe véve az ideális fejlődési tempódat. A felhasználók egy könnyen értelmezhető és vizuálisan áttekinthető tanulási felületet kapjanak, amely tartalmazza az elérhető modulokat és leckéket. A rendszer minden leckét a felhasználó szintjének és haladásának megfelelően jelenít meg a tanulási útvonalon, optimalizálva az új anyagok és az ismétlések arányát. A felhasználói élmény javítása érdekében az útvonal (sötét és világos módban is) könnyen navigálható.

2. fejezet

Relációs adatkezelő rendszerek

2.1. Relációs adatbázis

A relációs adatbázisok egy rendkívül logikus, jól felépített adatmodellre támaszkodó tárolási rendszert alkotnak. Ezekben a környezetekben az információkat átlátható, táblázatos formában – úgynevezett relációkban – rendszerezzük, ahol az oszlopok az egyes tulajdonságokat (attribútumokat), a sorok pedig a konkrét, egyedi adatrekordokat jelölik. A struktúra igazi erejét az adja, hogy a különálló táblák egyedi azonosítók (kulcsok) mentén precízen és dinamikusan összekapcsolhatók egymással.

Ezt a komplex adathálózatot olyan megbízható szoftverekkel hozzuk létre és menedzseljük, mint a MySQL, a PostgreSQL vagy az SQL Server. A bejegyzések módosítására és a célzott keresésekre a sztenderd SQL lekérdezőnyelvet alkalmazzuk. Ennek a megközelítésnek a legnagyobb előnye a szigorúan strukturált adattárolás, amely minden körülmények között garantálja az információk pontosságát, konzisztenciáját és sérthetetlenségét (integritását).

2.1.1. A relációs adatbáziskezelő rendszerek

A relációs adatbázis-kezelő rendszerek (RDBMS, azaz Relational Database Management System) olyan intelligens szoftverkörnyezetek, amelyek rácsos, táblázatos struktúrákba rendezve tartják karban az információkat. Minden egyes adattábla egyedi sorokból (konkrét bejegyzésekből) és oszlopokból (a bejegyzések tulajdonságaiból) épül fel. Ezekben a rendszerekben a különböző táblázatok nem elszigetelten működnek, hanem dedikált logikai relációkon, azaz szoros kapcsolatokon keresztül interakcióba lépnek egymással, garantálva az összefüggő adathalmazok átlátható kezelését.

E technológia legfőbb mozgatórugója a szabványosított SQL (Structured Query Language) lekérdezőnyelv, amellyel a tárolt rekordok villámgyorsan kinyerhetők, szerkeszthetők és menedzselhetők. A különálló adatcsomagokat speciális azonosítókkal – elsődleges (primary key) és idegen kulcsokkal (foreign key) – fűzzük össze egy egységes hálózattá. Az adatbázis normalizálásával sikeresen kiszűrjük a felesleges ismétlődéseket, így maximálisan megőrizhető a rendszer adatintegritása és konzisztenciája. Robusztus kialakításuknak, a hatékony tárhelykezelésnek és a gigantikus adatmennyiségek azonnali lekérdezhetőségének köszönhetően ezek az RDBMS technológiák ma már megkerülhetetlen alapkövei a modern üzleti és szoftveres infrastruktúráknak.

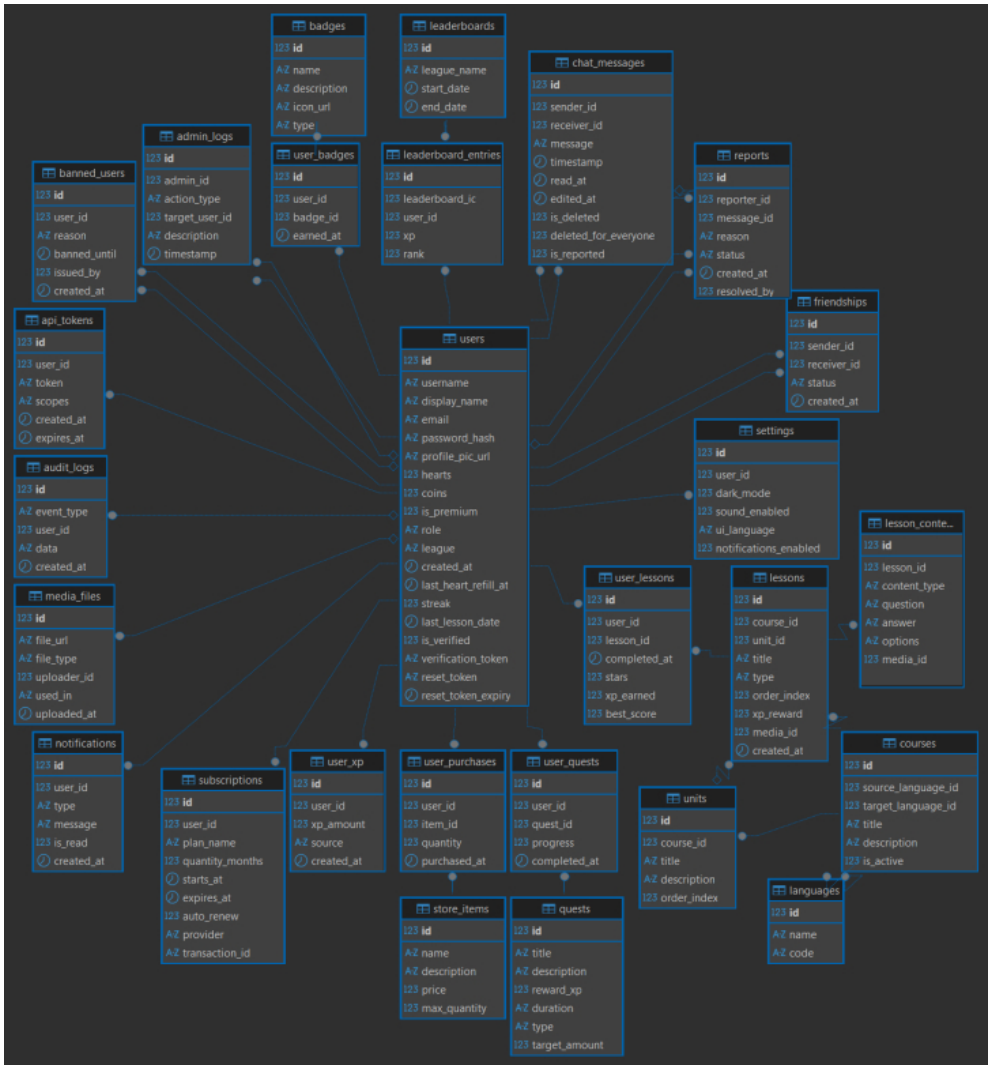
2.1.2. MySQL és MariaDB, valamint PHPMyAdmin

A MySQL és a belőle elágazó MariaDB a szoftverfejlesztés legelterjedtebb nyílt forráskódú relációs adatbázis-kezelői közé tartoznak. Technológiai alapjaik és működési logikájuk szinte teljesen megegyezik, hiszen a MariaDB közvetlenül a MySQL gyökereiből táplálkozik. Míg az eredetileg a Sun Microsystems által megalkotott MySQL ma már az Oracle vállalati égisze alatt működik, addig a MariaDB-t a platform eredeti megalkotói hívták életre, hogy hosszú távon is garantálják a projekt függetlenségét és szabad fejlődését.

Mindkét robusztus rendszer kiemelkedő teljesítményt, kiváló skálázhatóságot és gördülékeny kezelhetőséget nyújt, így tökéletes motorjai a modern, dinamikus webalkalmazásoknak. Natívan támogatják a fejlett tárolási motorokat (mint az InnoDB vagy a MyISAM), a komplex tranzakciókezelést, valamint a precízen finomhangolható, többszintű felhasználói és biztonsági mechanizmusokat.

Ezen adatbázisok kényelmes és hatékony adminisztrációjára a phpMyAdmin nyújtja a legnépszerűbb webes megoldást. Ez a nyílt forráskódú eszköz egy vizuális grafikus felülettel (GUI) váltja ki a bonyolult parancssori utasításokat, így a fejlesztők akár nyers SQL-kód írása nélkül is profi módon menedzselhetik a háttérrendszert.

Mivel a szoftver teljes egészében böngészőalapú, bármilyen eszközről azonnal elérhető. A felhasználók néhány kattintással építhetnek fel vagy módosíthatnak adattáblákat, kezelhetik a jogosultságokat, és futtathatnak egyedi lekérdezéseket. A platform hatalmas előnye, hogy kristálytisztán átláthatóvá teszi a struktúrákat: automatikusan generált diagramokon ábrázolja a relációkat, emellett zökkenőmentessé teszi az adatok tömeges importálását és exportálását. A beépített funkcióknak köszönhetően pedig a kritikus biztonsági mentések (backup) és az adat-helyreállítások is biztonságosan, pillanatok alatt elvégezhetők.



2.1. ábra. Relációs adatbázis

3. fejezet

React Fejlesztői Környezet

3.1. Bevezetés

A Meta által megalkotott és 2013-ban publikált React napjaink egyik legmeghatározóbb JavaScript-könyvtára, amely alapjaiban tette gördülékenyebbé az interaktív felhasználói felületek programozását. Megjelenése óta a modern webes technológiák, és különösen az egyoldalas webalkalmazások (Single Page Application, azaz SPA) fejlesztésének megkerülhetetlen alapkövévé vált.

Jelen szakdolgozat célkitűzése, hogy átfogóan elemezze a React-alapú fejlesztői ökoszisztémák felépítését. A kutatás részletesen górcső alá veszi az optimális munkakörnyezet kialakításához szükséges szoftveres eszközöket, illetve azokat a technológiai alapelveket, amelyek a hatékony, jól skálázható és modern alkalmazásfejlesztést garantálják.

3.2. React Fejlesztői Környezet

A sikeres szoftverfejlesztés nulladik lépése a megfelelő React ökoszisztéma precíz kialakítása. Ennek alapfeltétele a Node.js és a hozzá tartozó npm csomagkezelő telepítése, amelyek együttesen biztosítják a stabil JavaScript futtatókörnyezetet és a külső függőségek menedzselését. A projektek villámgyors inicializálásához a create-react-app segédprogram alkalmazása a legkézenfekvőbb, hiszen ez egy azonnal bevethető, előre konfigurált munkaállomást nyújt a programozóknak.

A technológia gerincét a komponensalapú architektúra adja, amely garantálja a szoftver moduláris felépítését és hosszú távú, átlátható karbantartását. A vizuális elemek és a logika ötvözését a JSX szintaxis teszi lehetővé: ez a HTML-szerű struktúrákat közvetlenül a JavaScriptbe ágyazza, drasztikusan javítva a kód ol-

vashatóságát. A komponensek belső állapotának (state) és a felülről érkező paramétereknek (props) a logikus kezelésével teljes kontrollt kapunk az alkalmazás dinamikus adatfolyama és a felhasználói interakciók felett. Ezen stabil alapok és a megfelelő munkakörnyezet lefektetésével minden akadály elhárul egy professzionális React-projekt elindítása elől.

3.2.1. React Telepítése és Projektindítás

A React-alapú fejlesztés megkezdésének elengedhetetlen feltétele a Node.js és a hozzá tartozó npm (Node Package Manager) csomagkezelő feltelepítése. Ezek a kulcsfontosságú szoftverek felelnek a külső JavaScript-könyvtárak és a komplex projektfüggőségek hatékony menedzseléséért. Amint ez a stabil szerveroldali futtatókörnyezet rendelkezésünkre áll, a create-react-app parancssori segédprogram alkalmazásával pillanatok alatt inicializálhatjuk és elindíthatjuk a legújabb alkalmazásunkat.

Az alábbi parancsokkal létrehozható egy új React projekt:

```
npx create-react-app my-app
cd my-app
npm start
```

A `create-react-app` gyorsan felállít egy alap React projektstruktúrát, amely a következő fájlokat tartalmazza:

- `src/` mappa: A projekt lelke és a tényleges forráskód otthona, amely magába foglalja a szoftver működéséhez elengedhetetlen összes állományt. Ebben a könyvtárban kapnak helyet a felhasználói felületet felépítő vizuális komponensek, valamint az adatkezelésért, az interakciókért és a specifikus funkciókért felelős üzleti logika is. A felületelemek jól strukturált, hierarchikus elrendezése biztosítja, hogy a fejlesztők gyorsan átlássák a rendszert, ami jelentősen megkönnyíti a jövőbeli karbantartást és a kódbázis bővítését.
- `public/` mappa: Ez a könyvtár a projekt statikus állományainak gyűjtőhelye, amelyeket a böngésző közvetlenül, a fordítási (build) folyamat beavatkozása és feldolgozása nélkül képes betölteni. Itt tároljuk azokat a fix, futásidőben változatlan erőforrásokat, mint például a képeket, az alkalmazás ikonjait, a beágyazott betűkészleteket és egyéb, a felülethez szükséges állandó médiafájlokat.
- `node_modules/`:Ebben a könyvtárban foglal helyet az alkalmazás működéséhez szükséges összes külső csomag és moduláris függőség, amelyeket az npm (Node Package Manager) telepít a projekt inicializálásakor. Ide kerülnek a futtatáshoz és a fejlesztéshez elengedhetetlen harmadik féltől származó szoftverek, mint maga a React könyvtár, a különböző segédcsomagok, valamint a tesztelési keretrendszerek és egyéb fejlesztői eszközök.

3.2.2. Fejlesztői Eszközök

A React fejlesztéséhez számos fejlesztői eszköz áll rendelkezésünkre, amelyek növelhetik a termelékenységet és a hibakeresés hatékonyságát.

React Developer Tools

A React Developer Tools egy hivatalos böngészőkiterjesztés, amely transzparens felületet biztosít a React-alapú szoftverek strukturális elemzéséhez és hatékony hibakereséséhez (debugging) közvetlenül a fejlesztői konzolban. Ezen az integrált környezeten belül a programozók kristálytisztán átláthatják a teljes vizuális architektúrát, feltérképezve a komponensek egymásra épülését és komplex hierarchikus viszonyait. Az eszköz egyik legnagyobb erőssége, hogy az egyes elemekhez rendelt bemeneti paraméterek (props) és belső állapotok (state) futásidőben, a felhasználói interakciókkal párhuzamosan monitorozhatók. A modern szoftverfejlesztés elengedhetetlen kellékei, a különféle Hook-ok – mint a useState, a useEffect vagy a useContext – működése szintén precízen nyomon követhető, így azonnali visszacsatolást kapunk az alkalmazás rejtett adatfolyamairól.

Mindemellett a kiegészítő egy fejlett profilozó (profiler) modullal is rendelkezik, amely a renderelési folyamatok teljesítménydiagnosztikájáért felel. Ezzel az eszközzel hajszálpontosan bemérhető, hogy mely komponensek, milyen sűrűséggel és pontosan mennyi idő alatt frissülnek a képernyőn. Ezek a funkciók együttesen drasztikusan felgyorsítják a szoftverhibák azonosítását és a kód optimalizálását, ezáltal a bővítmény használata megkerülhetetlen minden komolyabb React-projekt megvalósítása során.

3.3. React Alapjai

A React fundamentuma a komponensalapú architektúra, amely révén az egész szoftver apróbb, újrahasznosítható építőkockákból áll össze. Mivel minden egyes elem önállóan felel a saját belső állapotáért és működési logikájáért, a rendszer rendkívül rugalmasan bővíthető és menedzselhető. Ez a magas fokú modularitás drasztikusan megkönnyíti a forráskód karbantartását, hiszen az egyes egységek egymástól elszigetelten programozhatók, tesztelhetők és frissíthetők anélkül, hogy a módosítások veszélyeztetnék a platform globális stabilitását.

A funkciók logikai szeparációja és a modulok többszöri felhasználhatósága révén a programozók elkerülhetik a redundáns kódolást, ami jelentősen növeli a

termelékenységet. A komponensek szigorú izolációja a hibakeresést is leegyszerűsíti: azonnal és hajszálpontosan lokalizálható, melyik modul okozza a fellépő anomáliát. Ez a módszertan nem csupán a fejlesztési időt rövidíti le, hanem a csoportos munkavégzést is hatékonyan támogatja, mivel a csapattagok egyidejűleg, párhuzamos szálakon dolgozhatnak az eltérő elemeken. Ezzel a logikával a React garantálja, hogy a szoftver még masszív méretek és komplex funkciók integrálása esetén is maximálisan skálázható maradjon, miközben az alapok kristálytiszták és jól strukturáltak maradnak.

3.3.1. Komponensek és JSX

A React komponensek alapvetően olyan JavaScript függvények vagy osztályok, amelyek egy speciális, HTML-hez hasonló szintaxisbővítményt, az úgynevezett JSX-et (JavaScript XML) alkalmazzák. Ez a megoldás lehetővé teszi a fejlesztők számára, hogy a felület leíró kódját közvetlenül a JavaScript logikába ágyazzák, maradéktalanul megőrizve a nyelv teljes eszköztárát. A JSX használatával a forráskód nagyságrendekkel átláthatóbbá válik, hiszen az üzleti logika és a vizuális megjelenítés szorosan, egyetlen strukturált egységben kap helyet.

Fontos hangsúlyozni, hogy a JSX-ben megírt, HTML-nek tűnő kódsorok valójában nem nyers webes elemek; a fordítási folyamat során natív JavaScript kifejezésekké alakulnak, amelyeket a böngésző már képes értelmezni és hibátlanul végrehajtani. Ez a működési elv teszi igazán hatékonnyá a dinamikus tartalmak generálását. A szintaxis ugyanis megengedi, hogy a statikus elemek közé tetszőleges JavaScript kifejezéseket – például változókat, logikai feltételeket, függvényhívásokat vagy ciklusokat – ágyazzunk be, azonnal reagálva a felhasználói interakciókra.

Ennek az integrált megközelítésnek köszönhetően a szoftver minden rétege (az adatok, a vezérlés és a UI) tiszta és logikus rendszert alkot. A JSX-alapú komponensek maximálisan támogatják a kód újrahasznosíthatóságát, így a programozók gyorsan és ismétlődések nélkül építhetik fel a komplex felületeket. A React és a JSX fúziója végeredményben egy olyan robusztus alapot biztosít, amely garantálja, hogy az alkalmazás rugalmasan lekövesse a változó igényeket, miközben az architektúra hosszú távon is skálázható és karbantartható marad.

3.3.2. Állapotkezelés a React-ben

A dinamikus és interaktív felhasználói felületek (UI) megalkotásának legkritikusabb pillére a React komponensek belső állapotkezelése (state management). Az állapot, vagyis a state, egy olyan dedikált adatszerkezet, amely lehetővé teszi az egyes építőkockák számára az aktuális információk memóriában tartását. Ez a belső mechanizmus biztosítja a rendszer azonnali és reszponzív reakcióját a különböző felhasználói interakciókra – legyen szó egy egyszerű gombkattintásról, vagy akár komplex űrlapok adatainak valós idejű módosításáról.

A modern, funkcionális architektúrában a useState Hook nyújt letisztult, beépített interfészt ezen állapotok deklarálásához és manipulálásához. Amint egy állapot értéke megváltozik, a React motorja automatikusan újrarendereli (frissíti) az érintett komponenst, garantálva, hogy a vizuális felület mindig tökéletes szinkronban maradjon a háttérben futó üzleti logikával. Míg a kisebb léptékű, elszigetelt funkcióknál tökéletesen elegendő a komponensek szintjén tartott lokális állapotok használata, addig az összetettebb, többszintű adatáramlást (prop drilling) igénylő rendszereknél elengedhetetlen a globális állapotkezelő megoldások bevezetése. Ilyen esetekben a beépített Context API vagy az iparági sztenderdnek számító, robusztus Redux alkalmazása a legcélravezetőbb. Összességében a precízen megtervezett állapotkezelés az a központi mozgatórugó, amely irányítja a szoftver viselkedését, és alapjaiban határozza meg a zökkenőmentes felhasználói élmény (UX) minőségét.

useState Hook

A `useState` egy hook, amely lehetővé teszi az állapotkezelést a funkcionális komponensekben. Az alábbi példa egy egyszerű betöltő rendszert mutat be, amely a `useState` használatával kezeli az állapotot:

```
import React from 'react';
import './LoadingSpinner.css';

export default function LoadingSpinner({ size = 'medium',
    message = 'Loading...' }) {
  const sizeClasses = {
    small: 'spinner-small',
    medium: 'spinner-medium',
    large: 'spinner-large'
  };

  return (
    <div className="spinner-container">
      <div className={spinner ${sizeClasses[size]}}>
        <div className="spinner-ring"></div>
        <div className="spinner-ring"></div>
        <div className="spinner-ring"></div>
      </div>
      {message && <p className="spinner-message">{message
          }</p>}
    </div>
  );
}
```

3.3.3. Props

A React komponensek közötti kommunikációt az úgynevezett `props` (properties) segítségével valósítják meg. A `props` adatokat továbbítanak az egyik komponensből a másikba.

```
function Greeting(props) {
    return <h1>Hello, {props.name}</h1>;
}

function App() {
    return <Greeting name="React Developer" />;
}

export default App;
```

3.3.4. Komponensek Típusai

A React komponensek két fő típusa:

- **Funkcionális komponensek**: Ezek lényegében letisztult, transzparens JavaScript függvények. Működési logikájuk rendkívül egyszerű: bemeneti paraméterként fogadják a felülről érkező, csak olvasható adatokat (a props objektumot), majd eredményként a vizuális megjelenítést leíró JSX struktúrát adják vissza. A Hook-ok megjelenésével ezek a pehelykönnyű elemek képessé váltak az állapotkezelésre is, így napjainkban ez a leginkább preferált, modern fejlesztési forma.
- **Osztály alapú komponensek**: Hagyományos, objektumorientált (ES6) osztálystruktúrára épülő, robusztusabb elemek. A React korai szakaszaiban kizárólag ezek a komponensek tették lehetővé a komplex belső állapotkezelést (state) és a finomhangolt életciklus-metódusok (lifecycle methods) alkalmazását. Bár az új projektekben a funkcionális megközelítés már nagyrészt átvette a helyüket, a régebbi (legacy) rendszerek megértéséhez és karbantartásához továbbra is elengedhetetlen az ismeretük.

3.4. Profil Adatok Megváltoztatása

A kliensoldali (frontend) React alkalmazás szerves részét képezi egy dedikált profilkezelő modul, amelynek fókuszában a felhasználói adatok — kiemelten a személyes információk és jelszavak — biztonságos és gördülékeny módosítása áll. A fejlesztés elsődleges célkitűzése egy olyan interfész biztosítása volt, amely a CRUD (Create, Read, Update, Delete) műveletek során a legmagasabb szintű adatvédelmi protokollokat alkalmazza.

A folyamat során az alkalmazás end-to-end biztonsági szemléletet követ: az adatok nem csupán validálva vannak, hanem a hálózati kommunikáció során titkosított csatornán keresztül közlekednek, megakadályozva az érzékeny adatok (mint a jelszavak vagy személyes azonosítók) kompromittálódását.

Architekturális szempontból ez a komplex funkcionális egység az alábbi kulcsfontosságú React komponensek hierarchiájára épül:

3.4.1. Payload Komponens

A `Payload` a felhasználói profiladatok összeállításáért és a szerverre történő biztonságos továbbításáért felel. A logika dinamikusan kezeli a jelszómódosítást: csak akkor csatolja az érzékeny adatot a JSON-csomaghoz, ha a felhasználó ténylegesen megadott egy új értéket, egyébként eltávolítja azt a kérésből. A hálózati kommunikáció során a rendszer a modern fetch API-t és a Bearer Token alapú hitelesítést használja, biztosítva az adatok integritását és a jogosult hozzáférést.

```
const payload = {
  displayName: formData.displayName,
  username: formData.username,
  email: formData.email,
  password: formData.password
};

if (!payload.password) {
  delete payload.password;
}

const res = await fetch(PROFILE_URL, {
  method: 'PUT',
  headers: {
    Authorization: Bearer ${token},
    'Content-Type': 'application/json',
    Accept: 'application/json'
  },
  body: JSON.stringify(payload)
});
```

3.4.2. Új Profil Adatok

Az új megkapott adatok a Payload - on keresztül elérve biztonságosan elmenti.

```
Response body
{
  "id": 37,
  "username": "NEWnorbi",
  "displayName": "NEWnorbi",
  "email": "andren@kkszki.hu",
  "avatarUrl": null,
  "hearts": 2,
  "coins": 0,
  "isPremium": false,
  "createdAt": "márc. 2026",
  "totalXp": 0,
  "streak": 0
}
```

3.4.3. Felhasználói Élmény

A felhasználó egyszerűen és gyorsan tud jelszót változtatni a profilja oldalán ami biztosítja a jelszó újboli hashelését.

3.5. Összefoglalás

A modern webes ökoszisztémában a React fejlesztői környezetének és eszközrendszerének beható ismerete a hatékony és professzionális szoftverfejlesztési folyamat alapfeltétele. Dolgozatunkban részletesen feltártuk a környezet inicializálásának technikai lépéseit, a komponensalapú architektúra előnyeit, valamint a dinamikus adatkezelésért felelős állapotkezelési mechanizmusokat és a komponensek életciklusát szabályozó elveket.

A bemutatott fejlesztői segédeszközök és módszertanok együttesen olyan robusztus keretrendszert alkotnak, amely nem csupán a fejlesztési időt rövidíti le, hanem garantálja a kód hosszú távú fenntarthatóságát és skálázhatóságát is. A React ezen sokoldalú eszköztára teszi lehetővé, hogy a fejlesztők a technikai implementáció nehézségei helyett a valódi felhasználói értékteremtésre és az optimális élmény (UX) kialakítására fókuszálhassanak.

Köszönetnyilvánítás

Szeretnénk köszönetet mondani mindazoknak, akik segítették a vizsgaremekünk létrejöttét, külön megköszönve:

Négyesi Gábor tanár úrnak: *FontEnd - React - Szakmai Mentor*

Kerényi Róbert Nándor tanár úrnak: *BackEnd - Szakmai Mentor*

Irodalomjegyzék

[1] Meta Open Source: *React Documentations*,Quick Start. Elérhető: https://react.dev,(Letöltve: 2026. március 7.)

[2] Microsoft Azure Documentation: *Azure App Service Overview*,Elérhető: https://learn.microsoft.com/azure/app-service/ (Letöltve: 2026. március 7.)

[3] Node.js Foundation: *Node.js v20.x Documentation*,Elérhető: https://nodejs.org/docs/ (Letöltve: 2026. március 7.)